Estrutura de Dados

Curso Tecnologia em Análise de Desenvolvimento de Sistemas

Aula 08

Prof. Claudio Benossi

1.
Unidade

Vista da Prova

Esquenta para a Prova

Vamos fazer a nossa vista de prova!

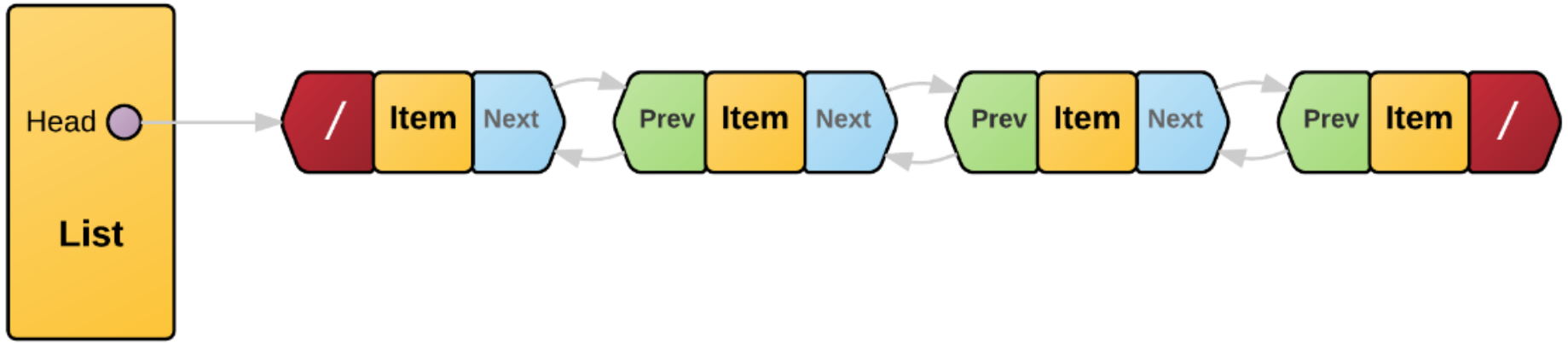
Individualmente!

1.
Unidade

Lista Duplamente Encadeada

Lista Encadeada

Uma lista encadeada é uma representação de uma sequência de objetos, todos do mesmo tipo, na memória RAM (= random access memory) do computador. Cada elemento da sequência é armazenado em uma célula da lista: o primeiro elemento na primeira célula, o segundo na segunda, e assim por diante.



Lista Duplamente Encadeada

Listas duplamente encadeadas

Cada item aponta tanto para o seu sucessor quanto para o antecessor;

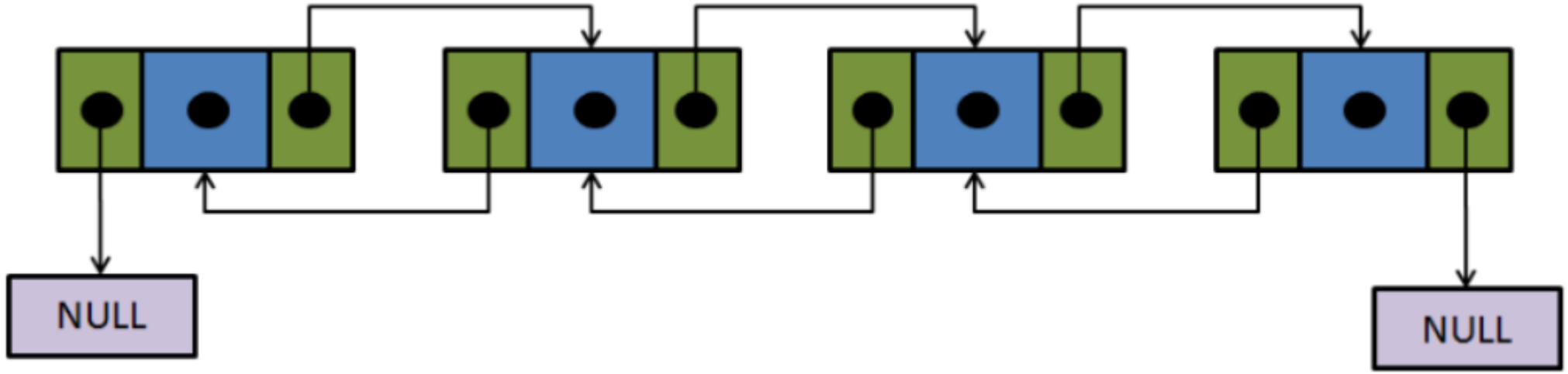
Lista Duplamente Encadeada

Para esse projeto vamos usar as seguintes classes:

```
class No {                  // Nó da lista duplamente encadeada
    int informacao;         // Informação que se quer armazenar
    No proximo;             // Ponteiro para o próximo nó
    No anterior;            // Ponteiro para o nó anterior
}

class ListaDupla {          // Lista duplamente encadeada
    No inicio;              // Ponteiro para o primeiro nó da lista
    No fim;                 // Ponteiro para o último nó da lista
    int tamanho;            // Quantidade de elemntos na lista
}
```

Lista Duplamente Encadeada



Implemente em linguagem de programação Java as seguintes operações na Lista Encadeada Dupla:

- Inserir no início
- Retirar do início;
- Inserir no meio;
- Retirar do meio;
- Inserir no fim;
- Retirar do fim;

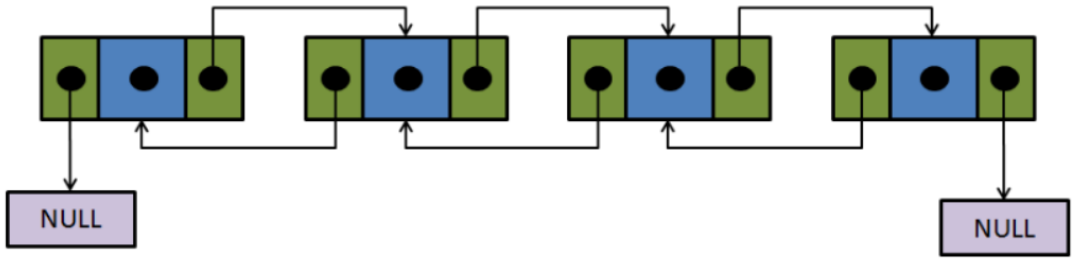
Lista Duplamente Encadeada

```
Class NoDuplo {          // Nó da lista duplamente encadeada
  String dado;           // Informação que se quer armazenar
  NoDuplo próximo;       // Ponteiro para próximo nó
  NoDuplo anterior;      // Ponteiro para o nó anterior
}


Class ListaDupla {       // Lista duplamente encadeada
  NoDuplo inicio;        // Ponteiro para o primeiro nó da lista
  NoDuplo fim;           // Ponteiro para o último nó da lista
}
```

Lista Duplamente Encadeada

Implemente em linguagem de programação Java
Dupla:




```
Escolha a opção:

1 -> Adicionar elemento
1.1 -> Adicionar elemento na posição específica
2 -> Excluir elemento na posição específica
3 -> Excluir elemento
4 -> Exibir o elemento Anteior (Antecessor)
5 -> Consultar a posição de um elemento
6 -> Consultar um elemento de uma posição
7 -> Consultar o tamanho da Lista
8 -> Exibir o primeiro elemento da Lista
9 -> Exibir o último elemento da Lista
10 -> Exibir o maior elemento da Lista
0 -> Exibir a Lista Completa
s -> Sair da Aplicação
```

Lista Duplamente Encadeada – Class NoDuplo

NoDuplo.java | ListaDuplaEncadeada.java | Exemplo_Lista_Duplamente_Encadeada.java

Source | History

```java
package exemplo_lista_duplamente_encadeada;

public class NoDuplo {
    public String dado;
    public NoDuplo proximo;
    public NoDuplo anterior;

    public NoDuplo() {
    }

    public NoDuplo(String dado) {
        this.dado = dado;
    }
}
```

NoDuplo								
anterior			Dado			proximo		
NoDplo			null			NoDplo		
anterior	Dado	Proximo				anterior	Dado	Proximo
	null						null	

Lista Duplamente Encadeada – Class ListaDuplaEncadeada

NoDuplo.java | ListaDuplaEncadeada.java | Exemplo_Lista_Duplamente_Encadeada.java

Source | History

```java
package exemplo_lista_duplamente_encadeada;

public class ListaDuplaEncadeada {
    private NoDuplo inicio;
    private NoDuplo fim;

    // Método construtor sem parâmetros - Objeto vazio
    public ListaDuplaEncadeada() {
    }

    // Método construtor com parâmetros - Objeto já com os dados
    public ListaDuplaEncadeada(NoDuplo inicio, NoDuplo fim) {
        this.inicio = inicio;
        this.fim = fim;
    }
```

ListaDuplaEncadeada					
inicio			fim		
NoDplo			NoDplo		
anterior	Dado	Proximo	anterior	Dado	Proximo
	Null			null	

Lista Duplamente Encadeada – Class ListaDuplaEncadeada

NoDuplo.java | ListaDuplaEncadeada.java | Exemplo_Lista_Duplamente_Encadeada.java

Source | History

```java
17      // /método de acesso aos atributos com escopo privado
18      public NoDuplo getInicio() {
19          return inicio;
20      }
21
22      public void setInicio(NoDuplo inicio) {
23          this.inicio = inicio;
24      }
25
26      public NoDuplo getFim() {
27          return fim;
28      }
29
30      public void setFim(NoDuplo fim) {
31          this.fim = fim;
32      }
```

Lista Duplamente Encadeada – Class ListaDuplaEncadeada

NoDuplo.java | ListaDuplaEncadeada.java | Exemplo_Lista_Duplamente_Encadeada.java

Source | History

```
34        // Método para exibir os dados da Lista Duplamente Encadeada
35        public void exibir() throws Exception {
36            System.out.println("LISTA -> ");
37            // criação de uma variável auxiliar
38            NoDuplo aux = inicio;
39            // Estrutura de repetição para percorrer a lista
40            while (aux != null){
41                System.out.print(aux.dado + " -> ");
42                aux = aux.proximo;
43            }
44            System.out.println("NULL\n");
45        }
```

Lista Duplamente Encadeada – Class ListaDuplaEncadeada

NoDuplo.java | ListaDuplaEncadeada.java | Exemplo_Lista_Duplamente_Encadeada.java

Source | History

```java
47      // Método para mostrar o tamanho da Lista
48      public int tamanhoLista() throws Exception {
49          NoDuplo aux = inicio;
50          int tamanho = 0;
51          while (aux != null){
52              tamanho ++;
53              System.out.println(aux.dado + " -> ");
54              aux = aux.proximo;
55          }
56          return tamanho;
57      }
```

Lista Duplamente Encadeada – Class ListaDuplaEncadeada

NoDuplo.java | ListaDuplaEncadeada.java | Exemplo_Lista_Duplamente_Encadeada.java

Source History

```java
    // este método insere os dados na seguencia ou seja no final da lista
    public void adicionar(String s) throws Exception {
        NoDuplo novoElemento = new NoDuplo(s);
        NoDuplo aux = inicio;
        // verifica se ele é o primeiro elemento da nossa lista
        if(inicio == null) {
            // sendo o primeiro elemento ele não tem próximo nem anterior
            novoElemento.proximo = null;
            novoElemento.anterior = null;
            // como ele é o primeiro ele vai ser o inicio e o fim da lista
            inicio = novoElemento;
            fim = novoElemento;
        } else {
```

Lista Duplamente Encadeada – Class ListaDuplaEncadeada

NoDuplo.java | ListaDuplaEncadeada.java | Exemplo_Lista_Duplamente_Encadeada.java

Source | History

```
72            } else {
73                // agora quando o elemento não for o primeiro da lista
74                // vamos percorrer a lista para achar a posição valida
75                while (aux.proximo != null){
76                    // vamos verificar se o elemento não é repetido
77                    if (aux.dado.equals(s)){
78                        throw new Exception("Elemento repetido");
79                    } else {
80                        aux = aux.proximo;
81                    }
82                }
83                // o ultimo elemento sempre deve ter o próximo como null
84                novoElemento.proximo = null;
85                aux.proximo = novoElemento;
86                novoElemento.anterior = aux;
87                fim = novoElemento;
88            }
89        }
```

NoDuplo.java | ListaDuplaEncadeada.java | Exemplo_Lista_Duplamente_Encadeada.java

Source History

```java
    // Método para INSERIR os dados na Lista Duplamente Encadeada
    // este método insere os dados posição determinada na lista
    public void adicionar(String s, int i) throws Exception {
        // verifica se a posição desejada é valida
        if(i > 0 || i > tamanhoLista()){
            throw new Exception("Posição Inválida!");
        }
        // novo elemento da nossa lista
        NoDuplo novoElemento = new NoDuplo(s);
        NoDuplo aux = inicio;
        // verifica se ele é vai na primeira posição da lista
        if(i == 0) {
            if(tamanhoLista() != 0){
                // caso já tenha um valor na primeira posição devemos mover os dados
                novoElemento.proximo = aux;
                aux.anterior = novoElemento;
                inicio = novoElemento;
                novoElemento.anterior = null;
            } else {
                // caso não ter nenhum elemento na lista ele será o primeiro
                inicio = novoElemento;
            }
        } else {
```

Lista Duplamente Encadeada – Class ListaDuplaEncadeada

NoDuplo.java | ListaDuplaEncadeada.java | Exemplo_Lista_Duplamente_Encadeada.java

Source History

```
113             } else {
114                 // vamos percorrer a lista para achar a posição desejada
115                 int contador = 1;
116                 while (contador < i){
117                     aux.proximo = novoElemento;
118                     contador ++;
119                     novoElemento.anterior = aux;
120                     fim = novoElemento;
121                 }
122                 novoElemento.proximo = aux.proximo;
123                 novoElemento.anterior = aux;
124                 aux.proximo = novoElemento;
125             }
126         }
```

Lista Duplamente Encadeada – Class ListaDuplaEncadeada

NoDuplo.java | ListaDuplaEncadeada.java | Exemplo_Lista_Duplamente_Encadeada.java

Source | History

```
128         // Método para remover elemento da Lista Encadeada
129         public String removeIndex(int i) throws Exception {
130             NoDuplo aux = inicio;
                NoDuplo lixo = null;
132             // verifica se o parâmetro informado é válida
133             if (i < 0 || i > tamanhoLista())
134                 throw new Exception("Indice inválido na Lista!");
135             if(tamanhoLista() == 0)
136                 throw new Exception("A lista está vazia!");
137             // verificar se o elemento a ser removido é o primeiro da lista
138             if (i == 0){
139                 lixo = aux;
140                 aux = aux.proximo;
141                 inicio = aux;
142             } else {
```

NoDuplo.java | ListaDuplaEncadeada.java | Exemplo_Lista_Duplamente_Encadeada.java

Source History

```
142             } else {
143                 // caso o elemento a ser removido não seja o primeiro
144                 int contador = 0;
145                 // percorrer a lista até achar a posição desejada
146                 while (contador < i){
147                     aux = aux.proximo;
148                     contador ++;
149                 }
150                 // armazena na variável lixo a informação a ser removida
151                 lixo = aux;
152                 aux.anterior.proximo = aux.proximo;
153                 if(aux != fim)
154                     aux.proximo.anterior = aux.anterior;
155                 else // caso ele seja o último elemento
156                     fim = aux;
157             }
158             // retorna o valor excluido
159             return lixo.dado;
160         }
```

Lista Duplamente Encadeada – Class ListaDuplaEncadeada

NoDuplo.java | ListaDuplaEncadeada.java | Exemplo_Lista_Duplamente_Encadeada.java

Source History

```java
162         // Método para remover o elemento de acordo com o conteúdo
163         public void removeElemento(String s) throws Exception {
164             // Verifica se tem elementos na Lista
165             if(tamanhoLista() == 0)
166                 throw new Exception("A lista está vazia!");
167             NoDuplo aux = inicio;
168             // criando uma variável lógica para controlar a exclusão
169             boolean excluir = false;
170             // vamos percorrera lista até achar o elemento
                while(aux != null && aux.dado != s){
172                 // se o objeto auxilar for igual ao valor passado pelo parâmetro
173                 if (aux.dado.equals(s)){
174                     if(aux == inicio){
175                         // verifica se é a primeira posição
176                         inicio = inicio.proximo;
177                         excluir = true;
178                     } else {
```

Lista Duplamente Encadeada – Class ListaDuplaEncadeada

NoDuplo.java | ListaDuplaEncadeada.java | Exemplo_Lista_Duplamente_Encadeada.java

Source History

```
178                     } else {
179                         // caso não seja a primeira posição
180                         aux.anterior.proximo = aux.proximo;
181                         aux.proximo = aux.anterior;
182                         excluir = true;
183                     }
184                 }
185                 aux = aux.proximo;
186             }
187             if (excluir == false)
188                 //caso não encontre o elemento
189                 throw new Exception("O elemento não foi localizado na Lista");
190         }
```

NoDuplo.java | ListaDuplaEncadeada.java | Exemplo_Lista_Duplamente_Encadeada.java

Source History

```java
192     // Método para mostrar o antecessor do elemento
193     public String antecessor(String s) throws Exception {
194         // verifica se tem elementos na lista
195         if(tamanhoLista() == 0)
196             throw new Exception("A lista está vazia!");
197         NoDuplo aux = inicio;
198         while(aux != null){
199             // verifica se o elemento está na lista
200             if(aux.dado.equals(s)){
201                 // verifica se o elemento é o primeiro da lista
202                 if(aux == inicio){
203                     throw new Exception("Primeiro elemento da Lista não tem Antecessor!");
204                 } else {
205                     return aux.anterior.dado;
206                 }
207             }
208             aux = aux.proximo;
209         }
210         throw new Exception("O elemento não foi localizado na Lista!");
211     }
```

NoDuplo.java | ListaDuplaEncadeada.java | Exemplo_Lista_Duplamente_Encadeada.java

Source History

```java
213         // Método para buscar a posição do elemento na Lista
214         public int posicao(String s) throws Exception {
215             // verifica se tem elementos na lista
216             if(tamanhoLista() == 0)
217                 throw new Exception("A lista está vazia!");
218             NoDuplo aux = inicio;
219             int contador = 0;
220             while(aux != null){
221                 if(aux.dado.equals(s)){
222                     return contador;
223                 } else {
224                     aux = aux.proximo;
225                     contador ++;
226                 }
227             }
228             throw new Exception("O elemento não foi localizado na Lista!");
229         }
```

NoDuplo.java | ListaDuplaEncadeada.java | Exemplo_Lista_Duplamente_Encadeada.java

Source History

```java
231     // Método que retorna o valor do elemento de acordo com a posição
232     public String BuscaPosicao(int i) throws Exception {
233         // verifica se tem elementos na lista
234         if(tamanhoLista() == 0)
235             throw new Exception("A lista está vazia!");
236         // verifica se o parâmetro informado é valido
237         if (i < 0 || i > tamanhoLista())
238             throw new Exception("Indice inválido na Lista!");
239         NoDuplo aux = inicio;
240         int contador = 0;
241         // percorrer a Lista até a posição informada
242         while(contador <= i){
243             if (contador == i){
244                 return aux.dado;
245             } else {
246                 aux = aux.proximo;
247                 contador ++;
248             }
249         }
250         return aux.dado;
251     }
```

Lista Duplamente Encadeada – Class ListaDuplaEncadeada

NoDuplo.java | ListaDuplaEncadeada.java | Exemplo_Lista_Duplamente_Encadeada.java

Source | History

```java
253     // Método para retornar o primeiro elemento da Lista
254     public String primeiroLista() throws Exception{
255         // verifica se tem elementos na lista
256         if(tamanhoLista() == 0)
257             throw new Exception("A lista está vazia!");
258         if(tamanhoLista() != 0)
259             return inicio.dado;
260         return null;
261     }
```

Lista Duplamente Encadeada – Class ListaDuplaEncadeada

NoDuplo.java | ListaDuplaEncadeada.java | Exemplo_Lista_Duplamente_Encadeada.java

Source History

```java
263         // Método para retornar o último elemento da Lista
264         public String ultimoLista() throws Exception{
265             // verifica se tem elementos na lista
266             if(tamanhoLista() == 0)
267                 throw new Exception("A lista está vazia!");
268             if(tamanhoLista() != 0)
269                 return fim.dado;
270             return null;
271         }
```

NoDuplo.java | ListaDuplaEncadeada.java | Exemplo_Lista_Duplamente_Encadeada.java

Source History

```java
273         // Método para retornar o maior valor da Lista
274         // Como nosso exemplo é uma lista de String
275         // ele retorna por ordem alfabética decrescente
276         public String maior() throws Exception {
277             // verifica se tem elementos na lista
278             if(tamanhoLista() == 0)
279                 throw new Exception("A lista está vazia!");
280             NoDuplo aux = inicio;
281             String maior = aux.dado;
282             // percorrer a Lista
283             while(aux != null){
284                 if (aux.dado.compareTo(maior) > 0){
285                     maior = aux.dado;
286                 } else {
287                     aux = aux.proximo;
288                 }
289             }
290             return maior;
291         }
292     }
```

Lista Duplamente Encadeada – Class teste

NoDuplo.java | ListaDuplaEncadeada.java | Exemplo_Lista_Duplamente_Encadeada.java

Source | History

```java
1   package exemplo_lista_duplamente_encadeada;
2
3   import java.util.Scanner;
4
    public class Exemplo_Lista_Duplamente_Encadeada {
6
7       public static void main(String[] args) {
8           Scanner entrada = new Scanner(System.in);
9           ListaDuplaEncadeada lista = new ListaDuplaEncadeada();
            String elemento = null;
            String opcao = null;
            int posicao = 0;
13
```

NoDuplo.java | ListaDuplaEncadeada.java | Exemplo_Lista_Duplamente_Encadeada.java

Source History

```
14          do{
15              System.out.println("\nEscolha a opção:\n");
16              System.out.println("1 -> Adicionar elemento ");
17              System.out.println("1.1 -> Adicionar elemento na posição específica ");
18              System.out.println("2 -> Excluir elemento na posição específica ");
19              System.out.println("3 -> Excluir elemento ");
20              System.out.println("4 -> Exibir o elemento Anteior (Antecessor) ");
21              System.out.println("5 -> Consultar a posição de um elemento ");
22              System.out.println("6 -> Consultar um elemento de uma posição ");
23              System.out.println("7 -> Consultar o tamanho da Lista ");
24              System.out.println("8 -> Exibir o primeiro elemento da Lista ");
25              System.out.println("9 -> Exibir o último elemento da Lista ");
26              System.out.println("10 -> Exibir o maior elemento da Lista ");
27              System.out.println("0 -> Exibir a Lista Completa ");
28              System.out.println("s -> Sair da Aplicação ");
29
30              opcao = entrada.next();
31
```

Lista Duplamente Encadeada – Class teste

NoDuplo.java | ListaDuplaEncadeada.java | Exemplo_Lista_Duplamente_Encadeada.java

Source | History

```
30          opcao = entrada.next();
31
            if(opcao.equals("1")){
33              System.out.println("\nInforme o valor do elemento");
34              elemento = entrada.next();
35              try {
36                  lista.adicionar(elemento);
37              } catch (Exception e){
                    e.printStackTrace();
39              }
40          }
```

Lista Duplamente Encadeada – Class teste

NoDuplo.java | ListaDuplaEncadeada.java | Exemplo_Lista_Duplamente_Encadeada.java

Source | History

```
40                  }
41                  else if(opcao.equals("1.1")){
42                      System.out.println("\nInforme o valor do elemento");
43                      elemento = entrada.next();
44                      System.out.println("\nInforme a posição do elemento");
45                      posicao = entrada.nextInt();
46                      try {
47                          lista.adicionar(elemento, posicao);
48                      } catch (Exception e){
                            e.printStackTrace();
50                      }
51                  }
```

Lista Duplamente Encadeada – Class teste

NoDuplo.java | ListaDuplaEncadeada.java | Exemplo_Lista_Duplamente_Encadeada.java

Source | History

```
51                  }
52                  else if(opcao.equals("2")){
53                      System.out.println("\nInforme a posição do elemento");
54                      posicao = entrada.nextInt();
55                      try {
56                          lista.removeIndex(posicao);
57                      } catch (Exception e){
                            e.printStackTrace();
59                      }
60                  }
```

Lista Duplamente Encadeada – Class teste

NoDuplo.java | ListaDuplaEncadeada.java | Exemplo_Lista_Duplamente_Encadeada.java

Source | History

```
61          else if(opcao.equals("3")){
62              System.out.println("\nInforme o valor do elemento");
63              elemento = entrada.next();
64              try {
65                  lista.removeElemento(elemento);
66              } catch (Exception e){
                    e.printStackTrace();
68              }
69          }
```

Lista Duplamente Encadeada – Class teste

NoDuplo.java | ListaDuplaEncadeada.java | Exemplo_Lista_Duplamente_Encadeada.java

Source | History

```
69                  }
70                  else if(opcao.equals("4")){
71                      System.out.println("\nInforme o valor do elemento");
72                      elemento = entrada.next();
73                      try {
74                          lista.antecessor(elemento);
75                      } catch (Exception e){
                            e.printStackTrace();
77                      }
78                  }
```

Lista Duplamente Encadeada – Class teste

NoDuplo.java | ListaDuplaEncadeada.java | Exemplo_Lista_Duplamente_Encadeada.java

Source History

```
78                  }
79                  else if(opcao.equals("5")){
80                      System.out.println("\nInforme o valor do elemento");
81                      elemento = entrada.next();
82                      try {
83                          lista.posicao(elemento);
84                      } catch (Exception e){
                            e.printStackTrace();
86                      }
87                  }
```

Lista Duplamente Encadeada – Class teste

NoDuplo.java | ListaDuplaEncadeada.java | Exemplo_Lista_Duplamente_Encadeada.java

Source History

```
87                  }
88                  else if(opcao.equals("6")){
89                      System.out.println("\nInforme a posição do elemento");
90                      posicao = entrada.nextInt();
91                      try {
92                          lista.BuscaPosicao(posicao);
93                      } catch (Exception e){
                            e.printStackTrace();
95                      }
96                  }
```

Lista Duplamente Encadeada – Class teste

NoDuplo.java | ListaDuplaEncadeada.java | Exemplo_Lista_Duplamente_Encadeada.java

Source History

```
96                  }
97                  else if(opcao.equals("7")){
98                      try {
99                          System.out.println("O tamanho da lista é: " + lista.tamanhoLista());
100                     } catch (Exception e){
                            e.printStackTrace();
102                     }
103                 }
```

Lista Duplamente Encadeada – Class teste

NoDuplo.java | ListaDuplaEncadeada.java | Exemplo_Lista_Duplamente_Encadeada.java

Source History

```
103                 }
104                 else if(opcao.equals("8")){
105                     try {
106                         lista.primeiroLista();
107                     } catch (Exception e){
                            e.printStackTrace();
109                     }
110                 }
```

Lista Duplamente Encadeada – Class teste

NoDuplo.java | ListaDuplaEncadeada.java | Exemplo_Lista_Duplamente_Encadeada.java

Source | History

```
110                 }
111                 else if(opcao.equals("9")){
112                     try {
113                         lista.ultimoLista();
114                     } catch (Exception e){
                            e.printStackTrace();
116                     }
117                 }
```

Lista Duplamente Encadeada – Class teste

NoDuplo.java | ListaDuplaEncadeada.java | Exemplo_Lista_Duplamente_Encadeada.java

Source | History

```
117                 }
118                 else if(opcao.equals("10")){
119                     try {
120                         lista.maior();
121                     } catch (Exception e){
                            e.printStackTrace();
123                     }
124                 }
```

Lista Duplamente Encadeada – Class teste

NoDuplo.java | ListaDuplaEncadeada.java | Exemplo_Lista_Duplamente_Encadeada.java

Source | History

```
124                 }
125                 else if(opcao.equals("0")){
126                     try {
127                         lista.exibir();
128                     } catch (Exception e){
                            e.printStackTrace();
130                     }
131                 }
```

Lista Duplamente Encadeada – Class teste

NoDuplo.java | ListaDuplaEncadeada.java | Exemplo_Lista_Duplamente_Encadeada.java

Source History

```
132                 else {
133                     System.out.println("Opção inválida!");
134                 }
135             }
                while(opcao != "s");
137         }
138     }
```

" Saber o que é correto e não fazer é falta de coragem”

Confúcio

Obrigado!

Se precisar ...

Prof. Claudio Benossi

claudio.benossi@sp.senac.br